

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:100

Annexure A

CASE STUDY

Code of Conduct:

I Monika.R(192324281) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Q. No	Understanding of Concepts (25)	Experimental Design (25)	Use of Tools (20)	Analysis of Results (20)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

CSA1402- COMPILER DESIGN

CO3-AT2

CASE STUDY

NAME: MONIKA R

REG.NO: 192324281

Submitted To:

Dr. N. Deepa

Submitted on:

12-08-2026

CASE STUDY

Set 8

1. A cloud computing company is developing a compiler for a distributed programming language that supports complex arithmetic and logical expressions. The compiler must construct syntax trees for efficient intermediate representation and perform semantic analysis using Syntax-Directed Definitions. During testing, incorrect attribute propagation causes errors in nested conditional statements and function calls. The compiler team decides to redesign the semantic rules using inherited and synthesized attributes. Analyze how S-attributed and L-attributed definitions can improve semantic evaluation and discuss which parsing strategy is more suitable for this compiler design.
2. A game development organization is implementing a compiler for a scripting language used in real-time animation systems. The compiler currently produces large parse trees that consume excessive memory and slow down code translation. To optimize performance, the development team proposes replacing parse trees with compact syntax trees and applying bottom-up evaluation methods. However, semantic actions related to datatype handling still produce inconsistent results. Examine how syntax tree construction and bottom-up semantic evaluation improve compiler efficiency and explain the role of attribute evaluation in reducing translation errors.
3. A healthcare software company is designing a compiler for a medical data processing language that supports integer, floating-point, and boolean operations. The semantic analyzer must ensure that all expressions and assignments satisfy datatype compatibility rules. During testing, programs containing mixed datatypes generate ambiguous semantic results due to improper type conversions. The compiler architect plans to introduce a strict type system with equivalence checking for type expressions. Investigate how type systems, type equivalence, and type conversion mechanisms contribute to reliable semantic analysis in this environment.
4. A financial analytics company is creating a translator that converts mathematical models into optimized intermediate code for high-speed execution. The translator uses top-down parsing and L-attributed definitions because inherited attributes are necessary for scope handling and parameter passing. However, semantic actions are executed before all attribute values become available, causing incorrect intermediate code generation. Analyze how top-down translation and proper attribute dependency management can resolve these issues and improve the correctness of syntax-directed translation.

5. A compiler testing team is evaluating the semantic behavior of a newly developed programming language used for scientific simulations. The compiler must perform type checking, detect incompatible expressions, and handle both implicit and explicit datatype conversions. During cross-platform execution, programs behave inconsistently because certain type conversions are interpreted differently by different systems. The team decides to redesign the semantic analyzer using stronger type equivalence and conversion rules. Evaluate how type checking, equivalence of type expressions, and controlled type conversion improve semantic reliability and portability across platforms.

ANSWERS:

1. Syntax-Directed Definitions using S-Attributed and L-Attributed Definitions

Case Study

A cloud computing company is developing a compiler for a distributed programming language that supports complex arithmetic and logical expressions. The compiler must construct syntax trees and perform semantic analysis using Syntax-Directed Definitions. During testing, incorrect attribute propagation causes errors in nested conditional statements and function calls. The compiler team decides to redesign the semantic rules using inherited and synthesized attributes.

Answer:

A Syntax-Directed Definition (SDD) associates attributes with grammar symbols and semantic rules with productions. These attributes help the compiler perform tasks such as type checking, syntax-tree construction, and intermediate-code generation.

There are two important types of attributes:

- S-Attributed Definition
- An S-attributed definition uses only synthesized attributes.
- A synthesized attribute is calculated from the attributes of the children of a parse-tree node.

Example:

$$E \rightarrow E1 + T$$
$$E.type = compatible(E1.type, T.type)$$

Here, the type of E is calculated from its child nodes E1 and T.

Advantages:

- Simple to implement.
- Suitable for bottom-up parsing.
- Works naturally with LR parsers.

Useful for expression evaluation and syntax-tree construction.

L-Attributed Definition

- An L-attributed definition can use both synthesized and inherited attributes.
- An inherited attribute obtains information from the parent or left-side siblings.

Example:

D → **T L**

L.in = T.type

Here, the type obtained from T is passed to L.

Advantages:

- Useful for passing contextual information.
- Suitable for declarations, function parameters, and nested scopes.
- Can be implemented using top-down parsing.

Comparison

Feature	S-Attributed	L-Attributed
Attributes	Only synthesized	Synthesized + inherited
Information flow	Bottom-up	Left-to-right
Suitable parser	LR / bottom-up	LL / top-down
Complexity	Simple	More flexible
Main use	Expression evaluation	Type propagation and declarations

Suitable Parsing Strategy

For this compiler, L-attributed definitions with top-down parsing are suitable when information needs to be passed from parent nodes to nested conditional statements and function calls. However, for purely expression-based semantic evaluation, S-attributed definitions with bottom-up parsing are simpler.

Conclusion

Using synthesized attributes for values produced by child nodes and inherited attributes for contextual information makes semantic analysis more reliable. Therefore, an L-attributed approach is more suitable when nested structures and contextual type information are involved.

2. Syntax Tree Construction and Bottom-Up Semantic Evaluation

Case Study

A game development organization is implementing a compiler for a scripting language used in real-time animation systems. The compiler currently produces large parse trees that consume excessive memory and slow down code translation. The development team proposes replacing parse trees with compact syntax trees and applying bottom-up evaluation methods.

Answer

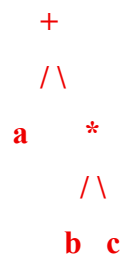
A parse tree represents every grammar symbol used during parsing. For complex programs, it can become very large and consume unnecessary memory.

A syntax tree, also called an abstract syntax tree (AST), removes unnecessary grammar symbols and keeps only the important program constructs.

For example:

Expression: **a + b * c**

The syntax tree can be represented as:



This is much smaller than the complete parse tree.

Benefits of Syntax Trees

1. Reduced memory usage
2. Faster semantic analysis
3. Faster intermediate-code generation
4. Easier optimization
5. Removes unnecessary grammar nodes

Bottom-Up Semantic Evaluation

In bottom-up evaluation, the compiler evaluates the attributes of child nodes first and then calculates the attribute of the parent.

For example: $E \rightarrow E1 + T$

The compiler first determines:

E1.type

T.type

then determines:

E.type

For: $3 + 5 * 2$

the compiler first evaluates:

$$5 * 2 = 10$$

and then:

$$3 + 10 = 13$$

Role of Attribute Evaluation

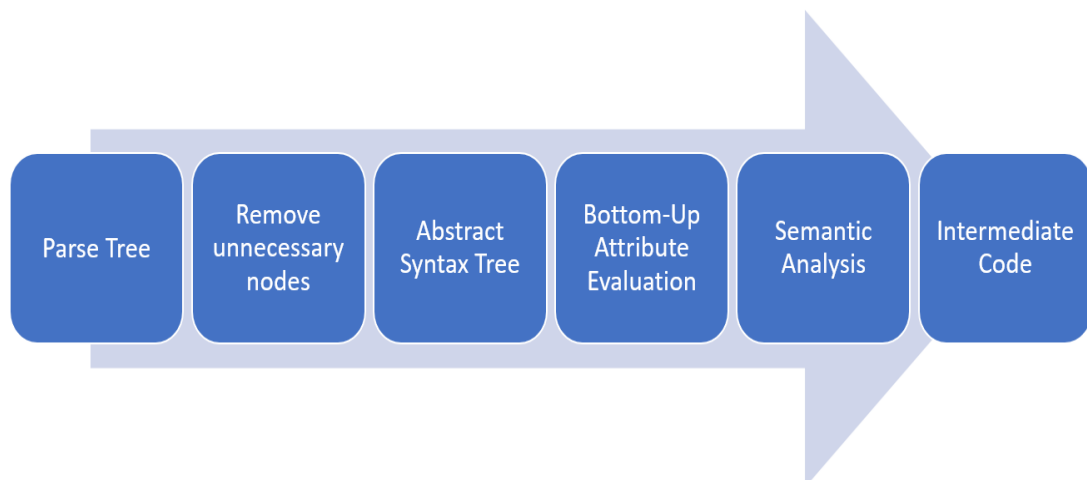
Attribute evaluation determines semantic information such as:

- Data types
- Variable values
- Symbol-table information
- Intermediate-code information
- Expression results

If attributes are evaluated in the wrong order, the compiler may use incomplete information and produce incorrect results.

How It Improves Efficiency

The combination of compact syntax trees and bottom-up semantic evaluation improves compiler efficiency because:



Conclusion

Replacing large parse trees with compact syntax trees reduces memory consumption and improves processing speed. Bottom-up semantic evaluation ensures that child attributes are calculated before their parent attributes, thereby reducing semantic translation errors.

3. Type Systems, Type Equivalence and Type Conversion

Case Study

A healthcare software company is designing a compiler for a medical data processing language that supports integer, floating-point, and Boolean operations. The semantic analyzer must ensure that all expressions and assignments satisfy datatype compatibility rules. During testing, programs containing mixed datatypes generate ambiguous semantic results due to improper type conversions.

Answer

A **type system** defines the rules that determine which data types can be used together in expressions, assignments, and function calls.

For example:

int a;

float b;

b = a + 2.5;

The compiler must determine whether the integer a can be combined with the floating-point value 2.5.

Type Checking

Type checking verifies that operations are performed between compatible types.

Example:

int + int is valid.

float + float is valid.

But: boolean + int is generally invalid.

The semantic analyzer detects such errors before code generation.

Type Equivalence

Type equivalence determines whether two types should be considered the same.

There are two common approaches:

1. Name Equivalence

Two types are equivalent only if they have the same declared type name.

Example:

type A = integer;

type B = integer;

Under strict name equivalence: $A \neq B$ because they have different names.

2. Structural Equivalence

Two types are equivalent if their structures are identical.

For example:

`A = record(int, float)`

`B = record(int, float)`

Under structural equivalence: $A = B$ because both have the same structure.

Type Expressions

A **type expression** represents the structure of a type.

Examples:

- integer
- float
- boolean

`array(10, integer)`

`pointer(integer)`

`function(integer  $\rightarrow$  float)`

The compiler can compare these type expressions to determine compatibility.

Type Conversion

Type conversion changes one datatype into another.

Implicit Conversion

The compiler automatically converts the type.

Example:

`int a = 10;`

`float b;`

`b = a;`

The compiler can convert: `int  $\rightarrow$  float`

Explicit Conversion

The programmer specifies the conversion.

Example:

`float x;`

`int y;`

`y = (int)x;`

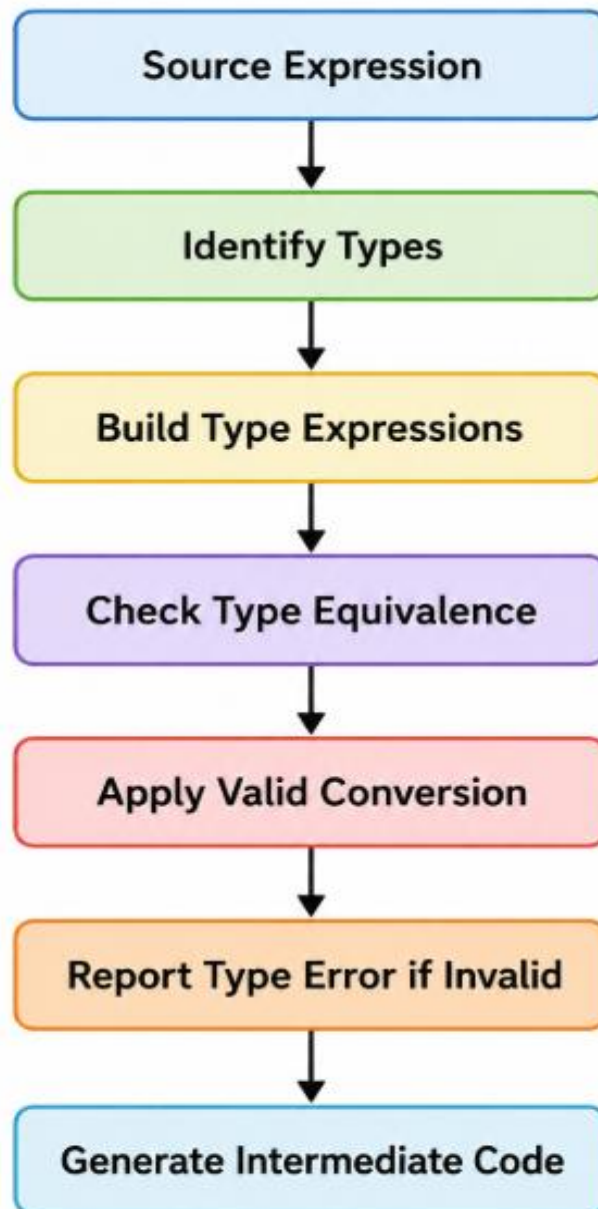
Conversion Hierarchy

A compiler can define controlled conversion rules such as: `integer  $\rightarrow$  float`

but should not automatically allow unsafe conversions such as:

`boolean  $\rightarrow$  complex object` unless explicitly permitted.

Recommended Type-Checking Process



Conclusion

A strict type system combined with type equivalence checking and controlled type conversion provides reliable semantic analysis. This is particularly important in healthcare applications because incorrect datatype interpretation can produce incorrect computational results.

4. Top-Down Parsing and L-Attributed Definitions

Case Study

A financial analytics company is creating a translator that converts mathematical models into optimized intermediate code for high-speed execution. The translator uses top-down parsing and L-attributed definitions because inherited attributes are necessary for scope handling and parameter passing. However, semantic actions are executed before all attribute values become available.

Answer

Top-down parsing starts from the start symbol and attempts to derive the input string by expanding non-terminals.

A common top-down parsing method is **LL parsing**.

L-attributed definitions are suitable for top-down parsing because inherited attributes can be evaluated from the parent and previously evaluated left siblings.

Problem with Incorrect Attribute Evaluation

Consider: $D \rightarrow T L$

where:

T = data type

L = list of variables

The type obtained from T must be passed to L. Therefore: $L.in = T.type$

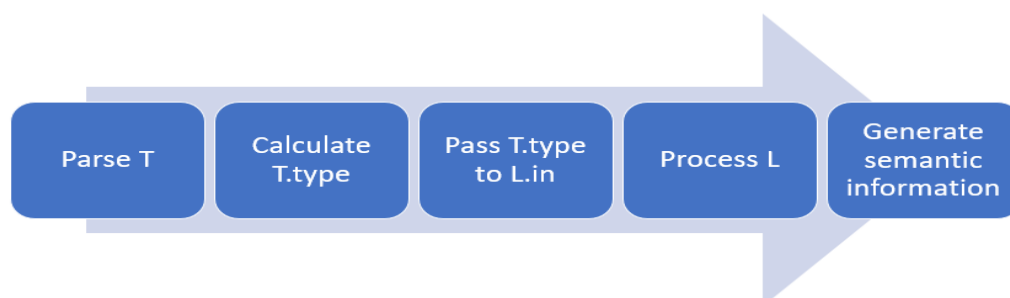
If the compiler performs the semantic action before calculating T.type, then L.in will contain incomplete or incorrect information.

This can cause:

- Incorrect variable types
- Incorrect scope information
- Incorrect parameter types
- Incorrect intermediate code

Correct Evaluation Order

The compiler should follow the dependency order:



For example:

int a, b, c;

The compiler first determines: $T.type = int$

Then: $L.in = int$

Therefore:

$a \rightarrow int$

$b \rightarrow int$

$c \rightarrow int$

L-Attributed Dependency

An L-attributed definition allows an inherited attribute of a symbol to depend on:

- Attributes of its parent.
- Attributes of symbols to its left.

This makes it suitable for left-to-right evaluation during top-down parsing.

Example

$D \rightarrow T L$

$L.in = T.type$

The dependency is:

$T.type$

↓

$L.in$

↓

Variable Types

Improving Correctness

The compiler should:

1. Construct the parse tree.
2. Identify attribute dependencies.
3. Evaluate attributes in dependency order.
4. Pass inherited attributes only after their required values are available.
5. Evaluate synthesized attributes after processing their children.
6. Generate intermediate code only after required semantic information is available.

Conclusion

Top-down parsing combined with properly ordered L-attributed definitions provides efficient syntax-directed translation. Correct dependency management prevents semantic actions from being executed before the required attribute values are available.

5. Type Checking, Type Equivalence and Controlled Type Conversion

Case Study

A compiler testing team is evaluating the semantic behavior of a newly developed programming language used for scientific simulations. The compiler must perform type checking, detect incompatible expressions, and handle both implicit and explicit datatype conversions. During cross-platform execution, programs behave inconsistently because certain type conversions are interpreted differently by different systems. The team decides to redesign the semantic analyzer using stronger type equivalence and conversion rules.

Answer

In a scientific simulation language, correct datatype handling is very important because calculations may involve integers, floating-point numbers, Boolean values, arrays, and other complex types.

The semantic analyzer must ensure that expressions and assignments are type compatible.

1. Type Checking

Type checking verifies whether operands and expressions use valid datatypes.

Example:

```
int a;  
int b;  
a = b + 10;
```

This is valid because both operands are integers.

But:

```
int a;  
boolean b;  
a = b + 10;
```

is invalid because Boolean and integer types are incompatible.

The compiler should produce a semantic error.

2. Strong Type Equivalence

The redesigned compiler should use **strong type equivalence** to avoid ambiguous interpretation.

For example:

```
type Meter = float;  
type Second = float;
```

Although both may have the same underlying representation, they can represent different concepts.

With strong equivalence, the compiler can prevent accidental operations such as: Meter + Second unless the language explicitly permits it. This improves semantic reliability.

3. Type Expressions

The compiler represents types using type expressions.

Examples:

integer

float

boolean

array(10, integer)

pointer(float)

function(integer $\rightarrow$ float)

Type expressions allow the semantic analyzer to compare complex types systematically.

For example:

array(10, integer)

can be compared with:

array(10, integer)

and identified as equivalent under the language's chosen equivalence rules.

4. Implicit Type Conversion

Implicit conversion is performed automatically by the compiler.

Example:

int x = 10;

float y;

y = x;

The compiler performs: **int** $\rightarrow$ **float** automatically.

However, implicit conversion should be allowed only when it is safe and clearly defined.

5. Explicit Type Conversion

Explicit conversion is requested by the programmer.

Example:

float x = 10.75;

int y;

y = (int)x;

The programmer explicitly requests:

float $\rightarrow$ int

The compiler should accept it according to the language rules and should handle possible loss of precision appropriately.

6. Conversion Rules

The compiler should maintain a well-defined conversion table.

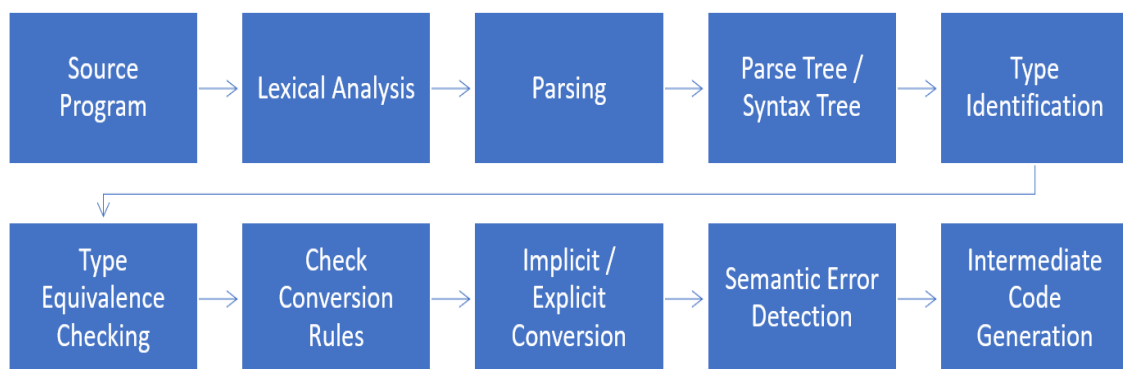
Example:

Source Type	Target Type	Conversion
Integer	Float	Allowed
Float	Integer	Explicit conversion preferred
Integer	Boolean	Not normally allowed
Boolean	Integer	Not normally allowed
Float	Float	No conversion
Integer	Integer	No conversion

This prevents different platforms from interpreting conversions differently.

7. Semantic Analyzer Design

The improved semantic analyzer can work as follows:



Example

Consider:

```
int a;  
float b;  
b = a + 2.5;
```

The compiler identifies:

$a \rightarrow \text{int}$

$2.5 \rightarrow \text{float}$

Then it applies the valid conversion: $\text{int} \rightarrow \text{float}$

The expression becomes conceptually: $(\text{float})a + 2.5$

Therefore, the assignment to b is valid.

Benefits of Strong Type Checking

Using stronger type equivalence and controlled conversion rules provides:

- Semantic reliability

- Early error detection
- Cross-platform consistency
- Prevention of incompatible operations
- Reduced ambiguity
- Better intermediate-code generation
- Improved portability

Conclusion

The compiler should use a strong type system, well-defined type equivalence rules, and controlled implicit and explicit conversions. These mechanisms ensure that scientific calculations behave consistently across different platforms and prevent semantic errors caused by ambiguous datatype conversions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

